

PROGRAMME FLUTTER PROFESSIONNEL – 50 HEURES

Public : Développeurs avec bases en algorithmique et POO

Fil rouge : Une application évolutive démarrée au module 2 et enrichie jusqu'au projet final.

Livrables stagiaires : Code source structuré, APK/AAB signé, rapport de tests.

MODULE 1 – Dart Essentiel (4 h)

Objectif : Maîtriser la syntaxe Dart indispensable au développement Flutter.

- **Théorie**
 - Variables, types, `final` / `const`, null safety (`?`, `!`, `??`, `??=`)
 - Fonctions, paramètres nommés, fonctions fléchées
 - Contrôle de flux, boucles
 - Collections : List, Set, Map et méthodes fonctionnelles (`map`, `where`, `fold`)
 - Classes, constructeurs, héritage, mixins
 - Asynchrone : `Future`, `async` / `await`
- **Travaux pratiques**
 - Console : gestionnaire de tâches (ajout, suppression, filtres, tri)
 - Simulation d'appels asynchrones avec `Future.delayed`
 - Sérialisation manuelle d'un objet en JSON

MODULE 2 – Flutter Fondamental (5 h)

Objectif : Installer l'environnement et comprendre l'architecture widget de Flutter.

- **Théorie**



- Flutter SDK, moteur de rendu, compilation JIT/AOT
- Installation, flutter doctor, émulateurs et appareils physiques
- Structure d'un projet (lib/, pubspec.yaml, main.dart)
- Philosophie « Everything is a widget »
- StatelessWidget vs StatefulWidget, cycle de vie
- Widgets de base
 - : Scaffold, AppBar, Text, Container, Row, Column, Icon, Image
- **Travaux pratiques**
 - Installation complète et première exécution
 - Carte de profil responsive
 - **Démarrage du fil rouge** : écran d'accueil statique avec liste factice

MODULE 3 – UI & Layout Responsive (6 h)

Objectif : Construire des interfaces adaptatives et modernes.

- **Théorie**
 - Mise en page avancée : Stack, Expanded, Flexible, Wrap
 - Listes optimisées : ListView.builder, GridView
 - Responsive design : MediaQuery, LayoutBuilder, OrientationBuilder
 - Thème global : ThemeData, Material Design 3, couleurs, typographie
 - Gestion des assets (images, polices, icônes)
- **Travaux pratiques (fil rouge)**
 - Création de la liste principale avec cartes personnalisées
 - Grille responsive (GridView) pour tablette
 - Adaptation portrait / paysage
 - Application d'un thème cohérent (couleur primaire, police)



MODULE 4 – Formulaires & Navigation (5 h)

Objectif : Rendre l'application interactive et structurer la navigation.

- **Théorie**
 - Saisies : `TextField`, `TextFormField`, `GlobalKey<FormState>`
 - Validation de formulaires (email, mot de passe, champs requis)
 - Composants : `DropDownButton`, `Checkbox`, `Switch`, `DatePicker`
 - Navigation : `Navigator`, routes nommées
 - `go_router` : routes déclaratives, passage de paramètres, redirections
- **Travaux pratiques (fil rouge)**
 - Écrans de connexion et d'inscription avec validation complète
 - Configuration de `GoRouter` pour 4 écrans (accueil, détail, formulaire, profil)
 - `SplashScreen` avec redirection conditionnelle

MODULE 5 – Gestion d'État avec Riverpod (6 h)

Objectif : Adopter une architecture robuste et séparer l'interface de la logique métier.

- **Théorie**
 - Limites de `setState`, état éphémère vs applicatif
 - Riverpod
 - `StateProvider`, `StateNotifierProvider`, `FutureProvider`, `ref.watch` / `ref.read`
 - `autoDispose`, gestion du cache
 - **Séparation précoce des responsabilités** : introduction des dossiers `models/`, `providers/`, `services/`, `repositories/` (Repository Pattern simplifié)
- **Travaux pratiques (fil rouge)**
 - Refactorisation complète du projet avec Riverpod



- Gestion d'une liste d'éléments avec ajout, suppression, modification réactive
- Recherche et filtres en temps réel
- Organisation du code selon les couches `models` / `providers` / `repositories` (avec un repository mocké pour le moment)

MODULE 6 – API REST & Persistance Locale (6 h)

Objectif : Connecter l'application à un back-end et garantir l'expérience offline.

- **Théorie**
 - Protocole HTTP, architecture REST, JSON
 - Package `dio` : requêtes, intercepteurs, gestion globale des erreurs
 - Sérialisation automatique : `json_serializable`, `freezed`
 - Stockage local : `SharedPreferences` (préférences), `Hive` (données structurées)
- **Travaux pratiques (fil rouge)**
 - **Code de départ fourni** : modèles Dart déjà générés via `json_serializable` pour que le stagiaire se concentre sur la logique réseau et le cache
 - Connexion à une API REST (fournie ou `json-server`)
 - CRUD complet via l'API
 - Mise en cache local avec `Hive` pour fonctionnement offline et synchronisation au démarrage

MODULE 7 – Authentification & Sécurité (4 h)



Objectif : Sécuriser l'accès à l'application avec JWT, première brique d'authentification professionnelle.

- **Théorie**
 - Authentification par JWT : login, register, token d'accès, refresh token
 - Stockage sécurisé : `flutter_secure_storage`
 - Intercepteurs `dio` pour injecter le token
 - Protection des routes avec `GoRouter` (redirection si non authentifié)
 - Déconnexion et nettoyage des données sensibles
- **Travaux pratiques (fil rouge)**
 - Implémentation du flux JWT complet (inscription, connexion, déconnexion)
 - Persistance du token et restauration de session au lancement
 - Routes protégées : accès au tableau de bord uniquement si authentifié
 - *(NB : ce module utilise une API REST classique. Le module suivant présentera Firebase Auth comme alternative.)*

MODULE 8 – Firebase (5 h)

Objectif : Intégrer un backend serverless moderne et proposer une alternative au couple API REST / JWT.

- **Théorie**
 - Configuration Firebase (`firebase_core`, `flutterfire_cli`)
 - Firebase Authentication : email/mot de passe, Google Sign-In
 - Cloud Firestore : collections, documents, requêtes, listeners temps réel
 - Firebase Storage : upload et récupération d'images
 - Aperçu de Firebase Cloud Messaging (notifications push)
- **Travaux pratiques (fil rouge)**



- **Choix guidé** : soit remplacer le flux JWT par Firebase Auth, soit ajouter Google Sign-In comme méthode complémentaire (un seul flux principal à implémenter dans le fil rouge pour éviter la redondance)
- Remplacement (ou duplication au choix) du CRUD API par Firestore
- Upload d'image de profil vers Firebase Storage
- Affichage des données en temps réel

MODULE 9 – Animations & Expérience Utilisateur (4 h)

Objectif : Créer une interface fluide, moderne et accessible.

- **Théorie**
 - Animations implicites
: `AnimatedContainer`, `AnimatedOpacity`, `AnimatedCrossFade`
 - Transitions entre écrans : `Hero`
 - Animations explicites
: `AnimationController`, `Tween`, `AnimatedBuilder` (bases)
 - Thème dynamique : bascule clair / sombre persistante
 - Accessibilité de base : `Semantics`, tailles de texte adaptatives
- **Travaux pratiques (fil rouge)**
 - Transition `Hero` entre la liste et le détail d'un élément
 - Micro-interaction sur le bouton d'ajout
 - Bascule thème clair/sombre avec persistance via `SharedPreferences`
 - Vérification du responsive sur plusieurs tailles d'écran

MODULE 10 – Architecture Professionnelle & Tests (4 h)

Objectif : Formaliser l'architecture en couches et garantir la qualité du code.



- **Théorie**
 - Clean Architecture simplifiée : séparation `presentation` / `domain` / `data`
 - Repository Pattern (rappel et approfondissement)
 - Injection de dépendances avec Riverpod
 - Tests unitaires : `package:test`, `mockito`
 - Tests de widgets : `flutter_test`, `WidgetTester`
- **Travaux pratiques (fil rouge)**
 - Réorganisation finale du projet pour correspondre à l'architecture cible (les bases posées aux modules 5 et 6 facilitent cette étape)
 - Écriture de tests unitaires pour un use case métier
 - Tests de widgets pour l'écran de connexion et la liste principale
 - Mock d'une API pour tester en isolation

MODULE 11 – Déploiement & Livraison (1 h)

Objectif : Préparer une version publiable de l'application.

- **Théorie**
 - Build release : APK vs AAB (Android), architecture iOS (aperçu)
 - Signature de l'application, gestion des keystores
 - Personnalisation : `flutter_launcher_icons`, `flutter_native_splash`
 - Processus de publication (Google Play Console, App Store Connect)
 - Introduction CI/CD : Codemagic, GitHub Actions
- **Travaux pratiques**
 - Configuration et génération d'un AAB signé pour le fil rouge
 - Personnalisation de l'icône et du splash screen
 - Vérification finale sur un appareil physique

PROJET FINAL – Synthèse & Évaluation (6 h)

Objectif : Mobiliser toutes les compétences pour livrer une application professionnelle.

- **Déroulement**

- 1 h : Cadrage final, répartition des tâches, dépôt Git
- 4 h : Développement encadré avec revues de code
- 1 h : Présentations orales, démonstration, évaluation

- **Critères d'évaluation**

Critère	Pondération
Fonctionnalités (auth, CRUD, persistance)	30 %
Architecture et qualité du code	25 %
UI/UX responsive et thème	20 %
Tests (unitaires + widgets)	15 %
Présentation orale et démonstration	10 %



RÉCAPITULATIF DES DURÉES

Module	Durée
1. Dart Essentiel	4 h
2. Flutter Fondamental	5 h
3. UI & Layout Responsive	6 h
4. Formulaires & Navigation	5 h
5. Gestion d'État (Riverpod)	6 h
6. API REST & Persistance	6 h
7. Authentification & Sécurité	4 h
8. Firebase	5 h
9. Animations & UX	4 h
10. Architecture & Tests	4 h
11. Déploiement	1 h
Projet Final	6 h
TOTAL	50 h